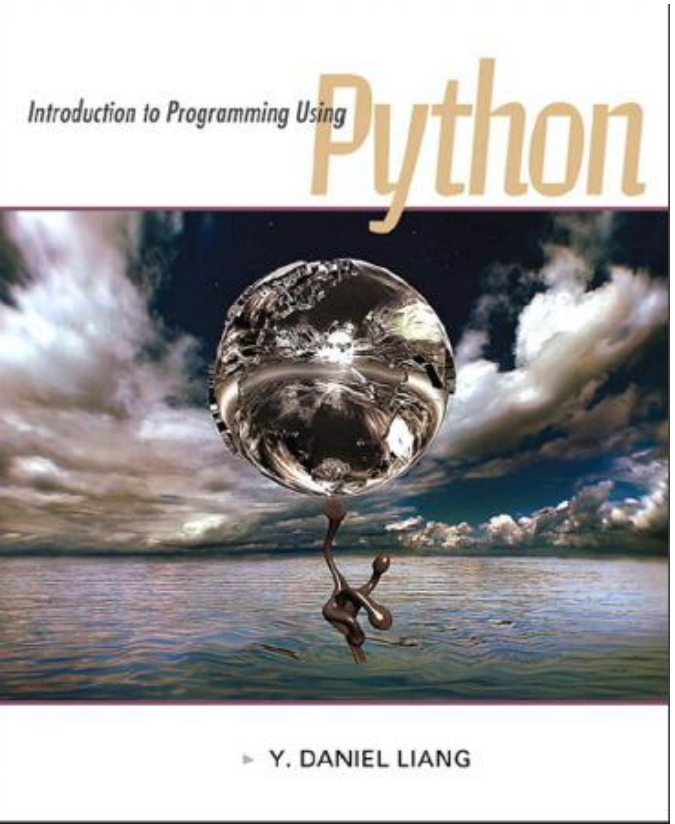


Chapter 03 Mathematical Functions, Strings, and Objects

Prepared By:

Eng. Eyad J. R. Elfarra

Objectives

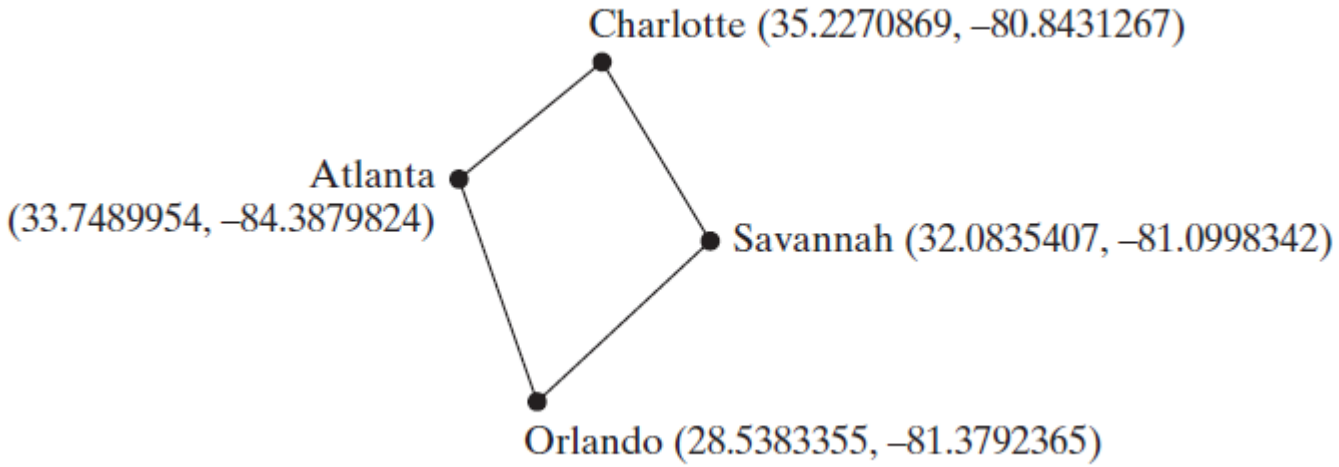
- To solve mathematics problems by using the functions in the **math** module (§3.2).
- To represent and process strings and characters (§§3.3-3.4).
- To encode characters using ASCII and Unicode (§§3.3.1-3.3.2).
- To use the **ord** to obtain a numerical code for a character and **chr** to convert a numerical code to a character (§3.3.3).
- To represent special characters using the escape sequence (§3.3.4).
- To invoke the **print** function with the end argument (§3.3.5).
- To convert numbers to a string using the **str** function (§3.3.6).
- To use the + operator to concatenate strings (§3.3.7).
- To read strings from the console (§3.3.8).
- To introduce objects and methods (§3.5).
- To format numbers and strings using the **format** function (§3.6).
- To draw various shapes (§3.7).
- To draw graphics with colors and fonts (§3.8).

2.1 Introduction

The focus of this chapter is to introduce functions, strings, and objects, and to use them to develop programs.

Motivations

- Suppose you need to estimate the area enclosed by four cities, given the GPS locations (latitude and longitude) of these cities, as shown in the following diagram. How would you write a program to solve this problem? You will be able to write such a program after completing this chapter.



3.2 Common Python Functions

Functions

- Python provides many useful **functions** for common programming tasks.
- A **function** is a group of statements that performs a specific task.
- **Python** provides a library of functions.
  - You have already used the functions:
  - eval, input, print, and int.
- These are **built-in** functions and they are **always available** in the Python interpreter.
- You **don’t** have to **import** any modules to use these functions.

Built In Functions

		Built-in Functions		
abs()	dict()	help()	min()	setattr()
all()	dir()	hex()	next()	slice()
any()	divmod()	id()	object()	sorted()
ascii()	enumerate()	input()	oct()	staticmethod()
bin()	eval()	int()	open()	str()
bool()	exec()	isinstance()	ord()	sum()
bytearray()	filter()	issubclass()	pow()	super()
bytes()	float()	iter()	print()	tuple()
callable()	format()	len()	property()	type()
chr()	frozenset()	list()	range()	vars()
classmethod()	getattr()	locals()	repr()	zip()
compile()	globals()	map()	reversed()	__import__()
complex()	hasattr()	max()	round()	
delattr()	hash()	memoryview()	set()	

Built In Functions

Function	*Description*	*Example*
`abs(x)`	Returns the absolute value for `x`.	`abs(-2)` is `2`
`max(x1, x2, ...)`	Returns the largest among `x1`, `x2`, ...	`max(1, 5, 2)` is `5`
`min(x1, x2, ...)`	Returns the smallest among `x1`, `x2`, ...	`min(1, 5, 2)` is `1`
`pow(a, b)`	Returns a^b. Same as `a ** b`.	`pow(2, 3)` is `8`

`round(x)`	Returns an integer nearest to `x`. If `x` is equally close to two integers, the even one is returned.	`round(5.4)` is `5`
		`round(5.5)` is `6`
		`round(4.5)` is `4`

`round(x, n)`	Returns the float value rounded to `n` digits after the decimal point.	`round(5.466, 2)` is `5.47` `round(5.463, 2)` is `5.46`

Examples

```
>>> abs(-3) # Returns the absolute value
3
>>> abs(-3.5) # Returns the absolute value
3.5
>>> max(2, 3, 4, 6) # Returns the maximum number
6
>>> min(2, 3, 4) # Returns the minimum number
2
>>> pow(2, 3) # Same as 2 ** 3
8
```

Examples

```
>>> pow(2.5, 3.5) # Same as 2.5 ** 3.5
24.705294220065465
>>> round(3.51) # Rounds to its nearest integer
4
>>> round(3.4) # Rounds to its nearest integer
3
>>> round(3.1456, 3) # Rounds to 3 digits after the decimal point
3.146
>>>
```

Math Module

- The Python **math** module provides the mathematical functions.
- Math module defines the following constants
  - **pi** and **e**
  - can be accessed using **math.pi** and **math.e**

The **math** Functions

Function	Description	Example
fabs(x)	Returns the absolute value of the argument.	fabs(-2) is 2
ceil(x)	Rounds x up to its nearest integer and returns this integer.	ceil(2.1) is 3 ceil(-2.1) is -2
floor(x)	Rounds x down to its nearest integer and returns this integer.	floor(2.1) is 2 floor(-2.1) is -3
exp(x)	Returns the exponential function of x (e^x).	exp(1) is 2.71828
log(x)	Returns the natural logarithm of x.	log(2.71828) is 1.0
log(x, base)	Returns the logarithm of x for the specified base.	log10(10, 10) is 1
sqrt(x)	Returns the square root of x.	sqrt(4.0) is 2
sin(x)	Returns the sine of x. x represents an angle in radians.	sin(3.14159 / 2) is 1 sin(3.14159) is 0
asin(x)	Returns the angle in radians for the inverse of sine.	asin(1.0) is 1.57 asin(0.5) is 0.523599
cos(x)	Returns the cosine of x. x represents an angle in radians.	cos(3.14159 / 2) is 0 cos(3.14159) is -1
acos(x)	Returns the angle in radians for the inverse of cosine.	acos(1.0) is 0 acos(0.5) is 1.0472
tan(x)	Returns the tangent of x. x represents an angle in radians.	tan(3.14159 / 4) is 1 tan(0.0) is 0
fmod(x, y)	Returns the remainder of x/y as double.	fmod(2.4, 1.3) is 1.1
degrees(x)	Converts angle x from radians to degrees	degrees(1.57) is 90
radians(x)	Converts angle x from degrees to radians	radians(90) is 1.57

Examples(code)

```
import math # import math module to use the math functions

# Test algebraic functions
print("exp(1.0) =", )
print("log(2.78) =", )
print("log10(10, 10) =", )
print("sqrt(4.0) =", )

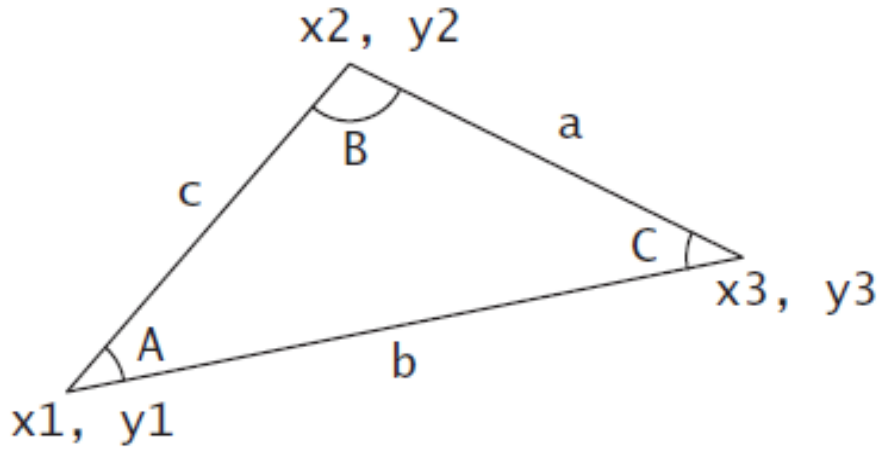
# Test trigonometric functions
print("sin(PI / 2) =", )
print("cos(PI / 2) =", )
print("tan(PI / 2) =", )
print("degrees(1.57) =", )
print("radians(90) =", math.radians(90))
```

Examples(Results)

```
exp(1.0) = 2.71828182846
log(2.78) = 1.0
log10(10, 10) = 1.0
sqrt(4.0) = 2.0
sin(PI / 2) = 1.0
cos(PI / 2) = 6.12323399574e-17
tan(PI / 2) = 1.63312393532e+16
degrees(1.57) = 89.9543738355
radians(90) = 1.57079632679
```

Problem: Compute Angles

- Given three points of a triangle, you can compute the angles using the following formula:




```
A = acos((a * a - b * b - c * c) / (-2 * b * c))
B = acos((b * b - a * a - c * c) / (-2 * a * c))
C = acos((c * c - b * b - a * a) / (-2 * a * b))
```

ComputeAngles.py

```
import math

x1, y1, x2, y2, x3, y3 = eval(input("Enter three points: "))

a = math.sqrt((x2 - x3) * (x2 - x3) + (y2 - y3) * (y2 - y3))
b = math.sqrt((x1 - x3) * (x1 - x3) + (y1 - y3) * (y1 - y3))
c = math.sqrt((x1 - x2) * (x1 - x2) + (y1 - y2) * (y1 - y2))
A = math.degrees(math.acos((a * a - b * b - c * c) / (-2 * b * c)))
B = math.degrees(math.acos((b * b - a * a - c * c) / (-2 * a * c)))
C = math.degrees(math.acos((c * c - b * b - a * a) / (-2 * a * b)))

print("The three angles are ", round(A * 100) / 100.0, round(B *
100) / 100.0, round(C * 100) / 100.0)
```

```
Enter three points: 1, 1, 6.5, 1, 6.5, 2.5 ↵Enter
The three angles are 15.26 90.0 74.74
```

Standard Modules

- Python has a large set of standard Modules
- When you download Python you download the standard modules
- https://docs.python.org/3/library/index.html
- Built-in Modules
  - cmath — Mathematical functions for complex numbers.
  - sqlite3 — DB-API 2.0 interface for SQLite databases
  - tkinter — Python interface to Tcl/Tk
  - turtle — Turtle graphics

3.3 Strings and Characters

Strings and Characters

- A string is a sequence of characters.
- String literals can be enclosed in matching single quotes (') or double quotes (").
- Python does not have a data type for characters.
  - So it treats characters and strings the same way.
- A single-character string represents a character.

```
letter = 'A' # Same as letter = "A"
numChar = '4' # Same as numChar = "4"
message = "Good morning"
# Same as message = 'Good morning'
```

NOTE

- For consistency, this book uses:
  - **Double quotes** for a **string** with **more than one character.**
  - A **single quotes** for a string with a **single character** or an **empty string**.
- This convention is **consistent** with **other** programming languages.
  - So, it will be **easy** to convert a Python program to a program written in other languages.

Unicode and ASCII Code

- **Character** is stored in a computer as a **sequence** of 0s and 1s.
- **Mapping** a character to its binary representation is called **character encoding**.
- There are different ways to encode a character, for example:
  - ASCII (American Standard Code for Information Interchange).
  - Unicode
- **ASCII** is a **7-bit** encoding scheme for representing all uppercase and lowercase letters, digits, punctuation marks, and control characters.
- **ASCII** uses numbers **0** through **127** to represent characters.

Appendix B: ASCII Character Set

- ASCII Character Set is a subset of the Unicode from \u0000 to \u007f

TABLE B.1 ASCII Character Set in the Decimal Index

	0	*1*	*2*	*3*	*4*	*5*	*6*	*7*	*8*	*9*
0	nul	soh	stx	etx	eot	enq	ack	bel	bs	ht
1	nl	vt	ff	cr	so	si	dle	dcl	dc2	dc3
2	dc4	nak	syn	etb	can	em	sub	esc	fs	gs
3	rs	us	sp	!	″	#	$	%	&	’
4	(	)	*	+	,	-	.	/	0	1
5	2	3	4	5	6	7	8	9	:	;
6	<	=	>	?	@	A	B	C	D	E
7	F	G	H	I	J	K	L	M	N	O
8	P	Q	R	S	T	U	V	W	X	Y
9	Z	[	\	]	^	_	‘	a	b	c
10	d	e	f	g	h	i	j	k	l	m
11	n	o	p	q	r	s	t	u	v	w
12	x	y	z	{	|	}	~	del		

ASCII Character Set, cont.

- ASCII Character Set is a subset of the Unicode from \u0000 to \u007f

TABLE B.2 ASCII Character Set in the Hexadecimal Index

	0	*1*	*2*	*3*	*4*	*5*	*6*	*7*	*8*	*9*	*A*	*B*	*C*	*D*	*E*	*F*
0	nul	soh	stx	etx	eot	enq	ack	bel	bs	ht	nl	vt	ff	cr	so	si
1	dle	dcl	dc2	dc3	dc4	nak	syn	etb	can	em	sub	esc	fs	gs	rs	us
2	sp	!	“	#	$	%	&	’	(	)	*	+	,	-	.	/
3	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6	‘	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7	p	q	r	s	t	u	v	w	x	y	z	{	|	}	~	del

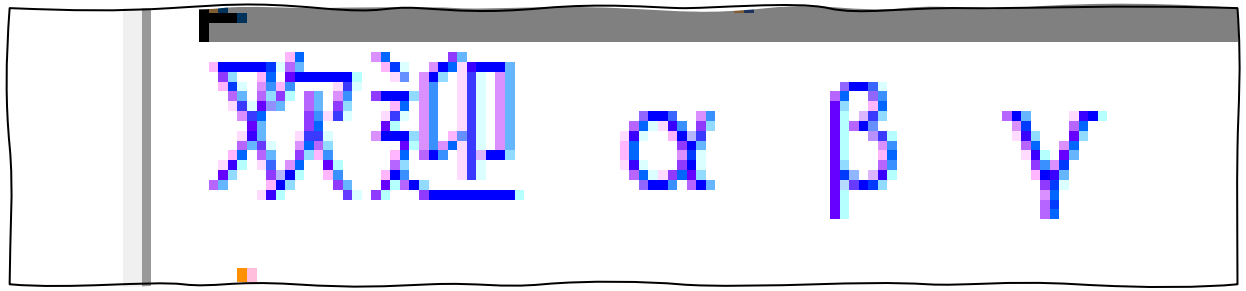
Unicode

- Python characters use *Unicode*, a 16-bit encoding scheme Python supports Unicode.
- Unicode is an encoding scheme for representing international characters.
- Why Unicode?
  - Because **ASCII** is a small subset of Unicode.
- A Unicode **starts** with \u, followed by **four hexadecimal** digits that run from \u0000 to \uFFFF.
- But Why **four hexadecimal** digits?

DisplayUnicode

- If **no Chinese** font is installed on your system, you will **not be able** to see the Chinese characters.

```
print("\u6B22\u8FCE \u03b1 \u03b2 \u03b3")
```



Functions **ord** and **chr**

- Python provides the **ord(ch)** function for **returning** the **ASCII code** for the character **ch.**
- The **chr(code)** function used for **returning** the **character** represented by the **code**.

```
>>> ch = 'a'
>>> ord(ch)
>>> 97
>>> chr(98)
>>> 'b'
```

Ord → diff capital letters, small letters

- ord(‘A’)
  - 65
- ord(‘a’)
  - 97
- ord(‘Aa’)??

```
>>> ord('a') - ord('A')
32
>>> ord('d') - ord('D')
32
>>> offset = ord('a') - ord('A')
>>> lowercaseLetter = 'h'
>>> uppercaseLetter =
 chr(ord(lowercaseLetter) - offset)
>>> uppercaseLetter
'H'
```

Escape Sequences for Special Characters

- Suppose you want to print the following statement using python:
  - He said, "John's program is easy to read”

```
print(“ He said, "John's program is easy to read “ ")
```



```
print("He said, "John's program is easy to read"")

SyntaxError: unterminated string literal (detected at line 1)
```

- What is the solution?
  - Use **escape sequence**
  - **\”**

```
print(“ He said, \"John's program is easy to read\“ ")
```

Escape Sequences for Special Characters

Description	*Escape Sequence*	*Unicode*
Backspace	`\b`	`\u0008`
Tab	`\t`	`\u0009`
Linefeed	`\n`	`\u000A`
Carriage return	`\r`	`\u000D`
Backslash	`\\`	`\u005C`
Single Quote	`\'`	`\u0027`
Double Quote	`\"`	`\u0022`

Escape Sequences for Special Characters

- New line **\n**: Moves text to a new line.

```
print("Hello\nWorld")
```



```
Hello
World
```

- Tab **\t**: Adds a horizontal tab (space)

```
print("Hello\tWorld")
```



```
Hello   World
```

- Carriage return **\r**: Moves cursor to the beginning of the line

```
print("Hello\rWorld")
```



```
World
```

- Backspace **\b**: Removes the previous character.

```
print("Helloo\b")
```

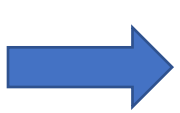


```
Hello
```

Escape Sequences for Special Characters

- Backslash ****: Prints a single backslash

```
print("This is a backslash: \\")
```



```
This is a backslash: \
```

- Single Quote **\'**: Used inside single-quoted strings.

```
print('It\'s a nice day')
```



```
It's a nice day
```

- Double Quote **\"** :Used inside double-quoted strings.

```
print("He said \"Hello\"")
```



```
He said "Hello"
```

Printing without the Newline

- **print** function automatically prints a linefeed **(\n)** to cause the output to advance to the next line.
- If you **don't want** this to, you can invoke the print function by passing a special argument **end = "anyendingstring"**
  - using the following syntax:

```
print(item, end = 'anyendingstring')
```

  - Example

```
print("AAA", end = ' ')
print("BBB", end = '')
print("CCC", end = '***')
print("DDD", end = '***')
```



```
AAA BBBCCC***DDD***
```

The str Function

- The str function can be used to convert a number into a string. For example,

```
>>> s = str(3.4) # Convert a float to string
>>> s
'3.4'
>>> s = str(3) # Convert an integer to string
>>> s
'3'
>>>
```

The String Concatenation Operator

- You can use the **+** operator to add two numbers.
- The **+** operator can also be used to **concatenate** (combine) two strings.
- Here are some examples:

```
>>> message = "Welcome " + "to " + "Python"
>>> message
'Welcome to Python'
>>> chapterNo = 2
>>> s = "Chapter " + str(chapterNo)
>>> s
'Chapter 2'
>>>
```

Reading Strings from the Console

- To read a string from the console, use the input function. For example, the following code reads three strings from the keyboard:

```
s1 = input("Enter a string: ")
s2 = input("Enter a string: ")
s3 = input("Enter a string: ")
print("s1 is " + s1)
print("s2 is " + s2)
print("s3 is " + s3)
```



```
Enter a string: Welcome ↵Enter
Enter a string: to ↵Enter
Enter a string: Python ↵Enter
s1 is Welcome
s2 is to
s3 is Python
```

3.4 Case Study: Minimum Number of Coins

Case Study: Minimum Number of Coins

- This program lets the **user enter the amount in decimal** representing **dollars and cents** and output a report listing the **monetary equivalent in single dollars, quarters, dimes, nickels, and pennies**.
- Your program should report maximum number of dollars, then the maximum number of quarters, and so on, in this order.

Coin	Front	Back	Value
Penny			1 ¢
Nickel			5 ¢
Dime			10 ¢
Quarter			25 ¢
Half-dollar			50 ¢
One-dollar			100 ¢

Steps(1/4)

1. Prompt the user to enter the amount as a decimal number, such as 11.56.

```
# Receive the amount
amount = eval(input("Enter an amount, for example, 11.56: "))
```

2. Convert the amount (11.56) into cents (1156).

```
# Convert the amount to cents
remainingAmount = int(amount * 100)
```

3. Divide the cents by 100 to find the number of dollars. Obtain the remaining cents using the cents remainder % 100.

```
# Find the number of one dollars
numberOfOneDollars = remainingAmount // 100
remainingAmount = remainingAmount % 100
```

Steps(2/4)

4. Divide the remaining cents by 25 to find the number of quarters. Obtain the remaining cents using the remaining cents remainder % 25.

```
# Find the number of quarters in the remaining amount
numberOfQuarters = remainingAmount // 25
remainingAmount = remainingAmount % 25
```

5. Divide the remaining cents by 10 to find the number of dimes. Obtain the remaining cents using the remaining cents remainder % 10.

```
# Find the number of dimes in the remaining amount
numberOfDimes = remainingAmount // 10
remainingAmount = remainingAmount % 10
```

Steps(3/4)

6. Divide the remaining cents by 5 to find the number of nickels. Obtain the remaining cents using the remaining cents remainder % 5.

```
# Find the number of nickels in the remaining amount
numberOfNickels = remainingAmount // 5
remainingAmount = remainingAmount % 5
```

7. The remaining cents are the pennies.

```
# Find the number of pennies in the remaining amount
numberOfPennies = remainingAmount
```

Steps(4/4)

8. Display the result.

```
# Display the results
print("Your amount", amount, "consists of\n",
    "\t", numberOfOneDollars, "dollars\n",
    "\t", numberOfQuarters, "quarters\n",
    "\t", numberOfDimes,  "dimes\n",
    "\t", numberOfNickels, "nickels\n",
    "\t", numberOfPennies, "pennies")
```

Results

```
Enter an amount, for example, 11.56: 11.56 ↵Enter
Your amount 11.56 consists of
     11 dollars
      2 quarters
      0 dimes
      1 nickels
      1 pennies
```

Caution: Floating-Point Precision Issue

- Converting a float to an integer can cause **loss** of precision.
- Example:
  - Input: 10.03
  - Calculation: 10.03 × 100 = 1002.999999999999
- Result:
  - Program may display 10 dollars and 2 pennies instead of 3 ❌
- ☑ Solution
  - Avoid floats for money calculations
  - Enter values as integers (in cents) instead
    - Example:Instead of 10.03 → use 1003

```
>>> 10.03*100
...
    1002.999999999999
>>>
```

3.5 Introduction to Objects and Methods

In Python, all data—including numbers and strings—are actually objects.

Introduction to Objects and Methods

- In Python, all data—including numbers and strings—are actually objects.
- An object is an entity.
- Each object has an **id** and a **type**.
- Objects of the same kind have the same type.
  - The **type** for the object is determined by Python according to the **value** of the object.
- You can use the **id** function and **type** function to get these information for an object.
- The **id** of the object:
  - The **id** for the object is **automatically** assigned a **unique integer** by Python **when the program is executed**.
  - The id for the object **will not be changed during the execution of the program**.
  - Python **may assign a different *id* every time the program is executed**.

Object Types and Ids

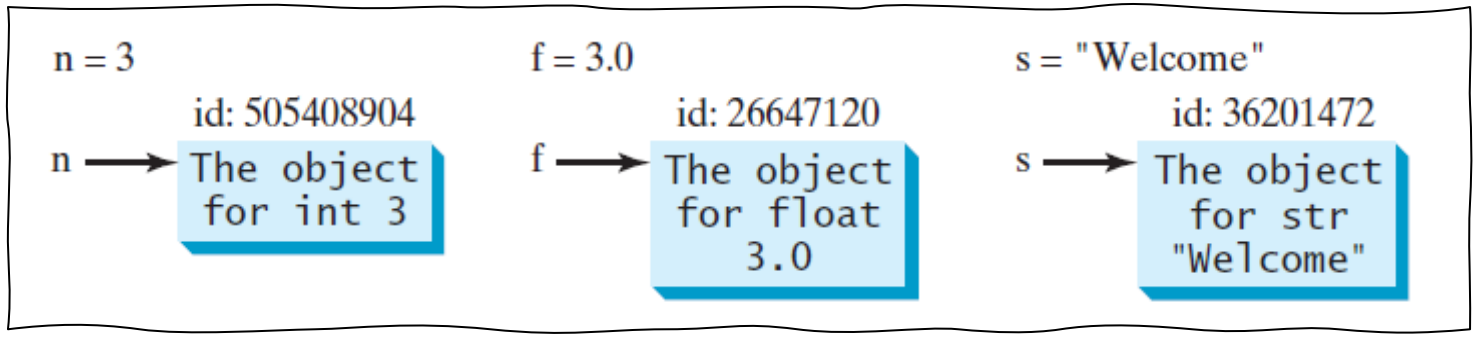
- The id and type functions are rarely used in programming, but they are good pedagogical tools for understanding objects.

```
>>> n = 3  # n is an integer
>>> id(n)
505408904
>>> type(n)
<class 'int'>
>>> f = 3.0  # f is a float
>>> id(f)
26647120
>>> type(f)
<class 'float'>
```

```
>>> s = "Welcome" # s is a string
>>> id(s)
36201472
>>> type(s)
<class 'str'>
```

OOP and str Objects

- The **id** and **type** functions are rarely used in programming, but they are good pedagogical tools for understanding objects.



Object vs. Object reference Variable

- For n = 3,
  - We say **n** is an **integer variable** that holds value 3.
  - Strictly speaking, n is a **variable** that references an **int object** for value 3.
  - For simplicity, it is fine to say n is an int variable with value 3.

Methods

- You can perform **operations** on an object.
- The operations are defined using **functions**.
- The **functions for the objects** are called *methods* in Python.
- Methods can **only be invoked** from a **specific object**.
- For example, the string type has the methods such as
  - **lower()** returns a new string in lowercase
  - **upper()**returns a new string in uppercase.
- What String methods are available?
  - https://docs.python.org/3/library/string.html
  - https://www.w3schools.com/python/python_ref_string.asp

str Object Methods

```
>>> s = "Welcome"
>>> s1 = s.lower() # Invoke the lower method
>>> s1
'welcome'
>>> s2 = s.upper() # Invoke the upper method
>>> s2
'WELCOME'
```

Striping beginning and ending Whitespace Characters

- Another useful string method is strip(), which can be used to **remove** (strip) the whitespace characters from **the both ends of a string**.
- The characters ' ', \t, \f, \r, and \n are known as the **whitespace characters**.

```
>>> s = "\t Welcome \n"
>>> s1 = s.strip() # Invoke the strip method
>>> s1
'Welcome'
```

3.6 Formatting Numbers and Strings

You can use the format *function to return a formatted string.*

Formatting Numbers and Strings

- Often it is desirable to display numbers in **certain format**.
- For example, the following code computes the interest, given the amount and the annual interest rate.

```
amount = 12618.98
interestRate = 0.0013
interest = amount * interestRate
print("Interest is", interest)
```



```
Interest is 16.404674
```

- The result has too many decimal places
- For currency, we usually need:
  - ☑ Only 2 decimal places

Solution

- Using round()

```
print("Interest is", round(interest, 2))
```



```
Interest is 16.4
```

  - Still not perfect → missing trailing zero (16.40)
- Using format() Function

```
print("Interest is", format(interest, ".2f"))
```



```
Interest is 16.40
```

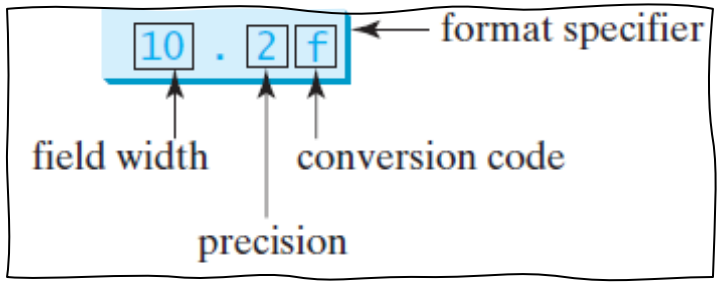
The format function

- The *format* function **formats** a number or a string and **returns** a string.

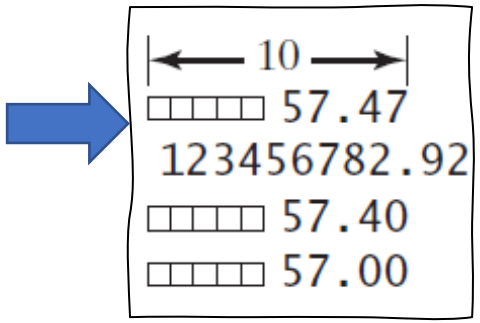
```
format(item, format-specifier)
```

Formatting Floating-Point Numbers

- If the item is a float value, you can use the specifier to give the width and precision of the format in the form of *width.precisionf*.
  - Width: specifies the width of the resulting string
  - Precision: specifies the number of digits after the decimal point
  - F: is called the conversion code




```
print(format(57.467657, "10.2f"))
print(format(123456782.923, "10.2f"))
print(format(57.4, "10.2f"))
print(format(57, "10.2f"))
```

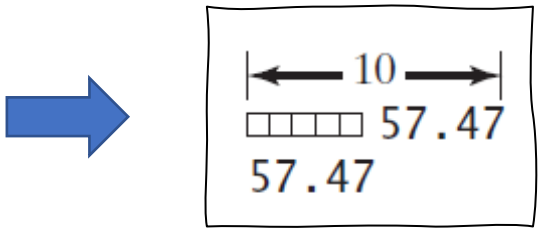


Where a square box [] denotes a blank space.
Note that the decimal point is counted as one space.

Formatting Floating-Point Numbers

- You can **omit** the width specifier.
  - If so, it **defaults** to **0**.
  - In this case, the width is **automatically** set to the size needed for formatting the number.

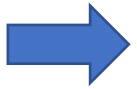
```
print(format(57.467657, "10.2f"))
print(format(57.467657, ".2f"))
```

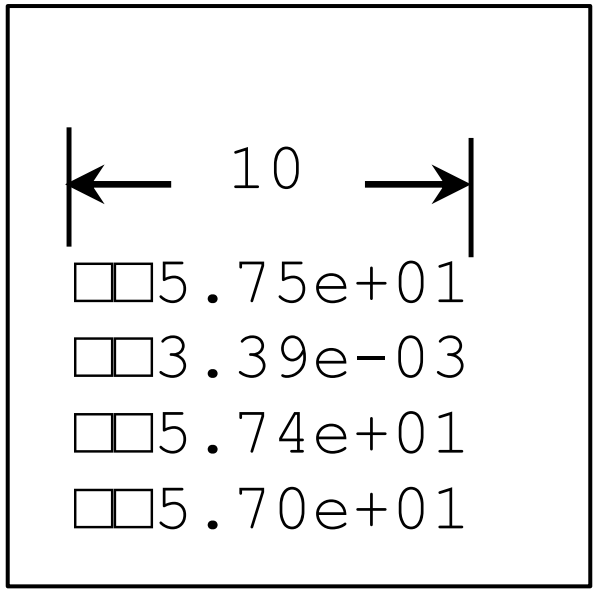


Formatting in Scientific Notation

- If you change the conversion code from f to e, the number will be formatted in scientific notation. For example,

```
print(format(57.467657, '10.2e'))
print(format(0.0033923, '10.2e'))
print(format(57.4, '10.2e'))
print(format(57, '10.2e'))
```

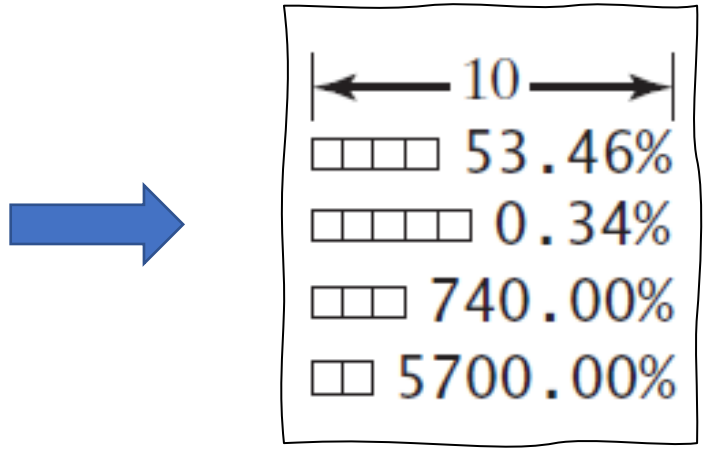




Formatting as a Percentage

- You can use the conversion code % to format numbers as a **percentage**. For example:

```
print(format(0.53457, '10.2%'))
print(format(0.0033923, '10.2%'))
print(format(7.4, '10.2%'))
print(format(57, '10.2%'))
```

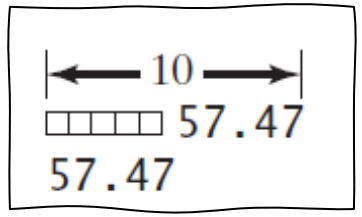


- The format 10.2% causes the number to be **multiplied by 100** and displayed with a **% sign** following it.
- The total width **includes** the % sign **counted as one space**.

Justifying Format

- By **default**, the format is **right** justified.
- You can put the symbol **<** in the format specifier to specify that the item is a **left** justified in the resulting format within the specified width.

```
print(format(57.467657, '10.2f'))
print(format(57.467657, '<10.2f'))
```



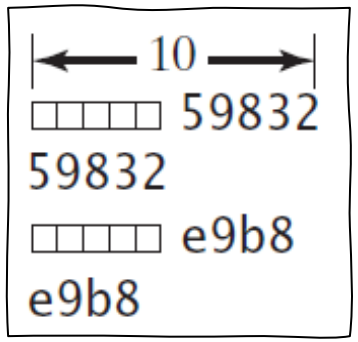

```
>>> print(format(57.467657, "10.2f"))
      57.47
>>> print(format(57.467657, ">10.2f"))
      57.47
>>> print(format(57.467657, "<10.2f"))
57.47
>>>
```

Formatting Integers

- You can use the conversion code **d**, **x**, **o**, and **b** to format an integer in **decimal, hexadecimal, octal, or binary**. You can specify a width for the conversion. For example,

```
print(format(59832, '10d'))
print(format(59832, '<10d'))
print(format(59832, '10x'))
print(format(59832, '<10x'))
```

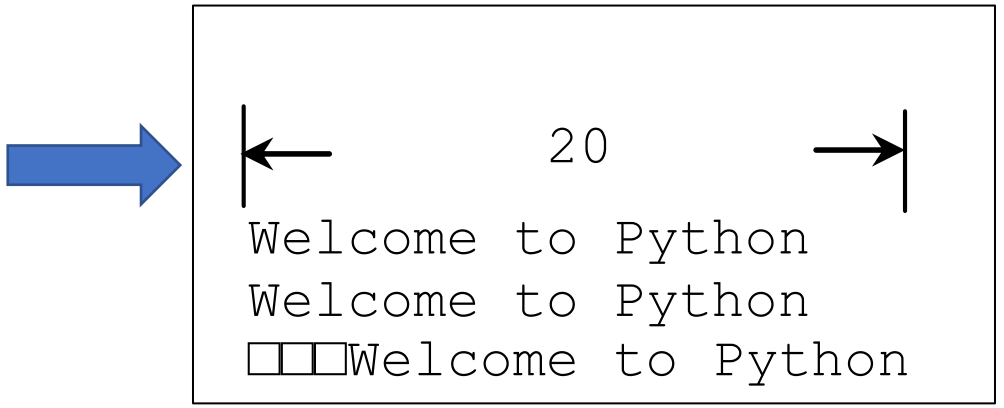




Formatting Strings

- You can use the conversion code s to format a **string** with a specified width. For example,

```
print(format("Welcome to Python", '20s'))
print(format("Welcome to Python", '<20s'))
print(format("Welcome to Python", '>20s'))
```



Formatting Strings

- But, what if the string size is greater than the width?

```
print(format("Welcome to Python and Java", ">20s"))
```

- Here the size of the message is 26 characters.
- In this case, the width is **automatically increased** to fit the string.

Results

```
>>> print(format("Welcome to Python and Java", ">20s"))
...
Welcome to Python and Java
>>>
```

Frequently used Specifiers

TABLE 3.4 Frequently Used Specifiers

Specifier	*Format*
`"10.2f"`	Format the float item with width 10 and precision 2.
`"10.2e"`	Format the float item in scientific notation with width 10 and precision 2.
`"5d"`	Format the integer item in decimal with width 5.
`"5x"`	Format the integer item in hexadecimal with width 5.
`"5o"`	Format the integer item in octal with width 5.
`"5b"`	Format the integer item in binary with width 5.
`"10.2%"`	Format the number in decimal.
`"50s"`	Format the string item with width 50.
`"<10.2f"`	Left-justify the formatted item.
`">10.2f"`	Right-justify the formatted item.

End of Chapter 03